

华中科技大学

2024

系统能力培养 课程实验报告

题目: RISC-V 模拟器开发

专业: 计算机科学与技术

班级: CS2110班

学号: U202012663

姓名: 李宝宁

电话: 15075117667

邮件: 2712477103@qq.com

完成日期: 2025-1-15



计算机科学与技术学院

# 目录

|                                        |           |
|----------------------------------------|-----------|
| <b>1 PA0-世界诞生的前夜：开发环境配置 .....</b>      | <b>1</b>  |
| 1.1 安装虚拟机 .....                        | 1         |
| 1.2 实验环境 .....                         | 1         |
| <b>2 PA1 - 开天辟地的篇章：最简单的计算机 .....</b>   | <b>2</b>  |
| 2.1 简易调试器基础 .....                      | 2         |
| 2.2 表达式求值 .....                        | 3         |
| 2.3 监视点 .....                          | 4         |
| <b>3 PA2 - 简单复杂的机器：冯诺依曼计算机系统 .....</b> | <b>6</b>  |
| 3.1 运行第一个 C 程序 .....                   | 6         |
| 3.2 实现更多的指令 .....                      | 7         |
| 3.3 实现输入输出，运行打字小游戏 .....               | 9         |
| <b>4 PA3 - 穿越时空的旅程：批处理系统 .....</b>     | <b>12</b> |
| 4.1 实现自陷操作及其过程 .....                   | 12        |
| 4.2 实现用户程序的加载和系统调用 .....               | 13        |
| 4.3 实现文件系统 .....                       | 15        |
| 4.4 运行仙剑奇侠传 .....                      | 16        |
| <b>5 总结与心得 .....</b>                   | <b>19</b> |

# 1 PA0-世界诞生的前夜：开发环境配置

## 1.1 安装虚拟机

下载课程组提供的 virtualbox 安装包以及虚拟机硬盘文件，virtualbox 安装完成后打开使用，新建虚拟机并导入下载的 vdi 文件。启动虚拟机，安装实验所需的开发工具。

## 1.2 实验环境

- (1) CPU 架构: x64
- (2) 平台: VirtualBox+Ubuntu22.04 LTS
- (3) 操作系统: GNU / LINUX
- (4) 编译器: gcc
- (5) 编程语言: c 语言



图 1.1 ubuntu 平台配置

## 2 PA1 - 开天辟地的篇章：最简单的计算机

PA1 的主要内容是实现一个 一个简易的 gdb，功能如下：

- (1) `help`: 打印命令的帮助信息
- (2) `c`: 继续运行被暂停的程序
- (3) `q`: 退出 NEMU
- (4) `si[N]`: 让程序单步执行 `N` 条指令后暂停执行，当 `N` 没有给出时，缺省为 1
- (5) `info SUBCMD`: `info r` 打印寄存器状态；`info w` 打印监视点信息
- (6) `p EXPR`: 求出表达式 `EXPR` 的值，`EXPR` 支持的运算请见调试中的表达式求值小节
- (7) `x N EXPR`: 求出表达式 `EXPR` 的值，将结果作为起始内存地址，以十六进制形式输出连续的 `N` 个 4 字节
- (8) `w EXPR`: 当表达式 `EXPR` 的值发生变化时，暂停程序执行
- (9) `d N`: 删除序号为 `N` 的监视点

### 2.1 简易调试器基础

要实现一个简易调试器，满足上面的要求，首先要能实现节点的单步执行以及每一步的节点信息的搜集和输出：

(1) 单步执行：在 `ui.c` 文件中，我们可以定义一个函数 `cmd_si`，读出跟在 `si` 后的数字，调用 `strtok` 函数读出数字，使用 `atoi` 函数将 `args` 转换为整数 `n`，然后调用 `cpu_exec` 函数执行 `n` 步指令。

(2) 打印寄存器状态：在 `ui.c` 文件中，我们可以定义一个函数 `cmd_info`，若读出的参数为“`r`”，则表名需要打印出寄存器状态。在 `reg.c` 文件中，补充 `isa_reg_display` 函数的内容，在 `cmd_info` 函数中调用 `isa_reg_display` 函数，显示寄存器的状态。

(3) 扫描内存：在 `ui.c` 文件中，定义了一个函数 `cmd_x`，其参数 `args` 包含两个参数：扫描的内存数量 `N` 与扫描内存的起始地址，使用 `expr` 函数解析，使用宏循环读取起始地址后的 `N` 个 4 字节数据。

测试结果如下图所示：

```
Welcome to riscv32-NEMU!
For help, type "help"
(nemu) si 2
80100000: b7 02 00 80          lui 0x80000,t0
80100004: 23 a0 02 00          sw 0(t0),$0
```

图 2.1 单步执行

```
(nemu) info r
---information of reg---
$0 = 0x00000000
ra = 0x00000000
sp = 0x00000000
gp = 0x00000000
tp = 0x00000000
t0 = 0x80000000
t1 = 0x00000000
t2 = 0x00000000
s0 = 0x00000000
s1 = 0x00000000
a0 = 0x00000000
a1 = 0x00000000
a2 = 0x00000000
```

图 2.2 打印寄存器指令

```
Welcome to riscv32-NEMU!
For help, type "help"
(nemu) x 4 $pc
80100000
0x80100000 : 0x800002b7
0x80100004 : 0x0002a023
0x80100008 : 0x0002a503
0x8010000c : 0x0000006b
```

图 2.3 扫描内存指令

## 2.2 表达式求值

表达式求值是一个经典的问题，要处理这个问题，需要将其拆分为两方面，先识别出表达式中的各个单元，然后再根据表达式的归纳定义进行递归求值

(1) 添加 token，即可能出现在表达式中的所有的 token：在 `expr.c` 文件中，就是运算符，比如+、-、\*、/、==、!=等等。

(2) 解析token：我们需要实现一个函数`make_token`，它的功能是根据表达式中 token 的规则，是解析字符串中的token，并将每个token的类型记录在`tokens`数组中。

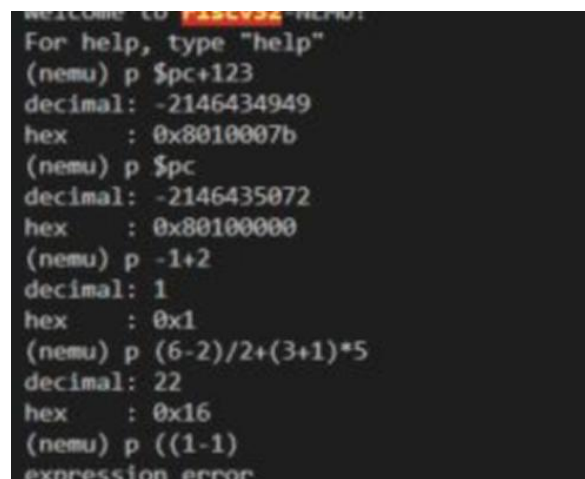
(3) 表达式求值 `expr`：根据 `tokens` 数组计算表达式的值，还需要再自行定义一些辅助运算的函数：如检查括号匹配的函数 `check_parentheses()`，此函数检查

表达式中的括号是否正确使用；要有定义运算符的优先级函数 `operator_priority()`，乘除优先级要高于加减；确定主运算符函数 `main_operator()`，确定计算符。

(4) 表达式计算函数 `eval`：定义一个函数 `eval`，接受两个参数 `p` 和 `q`，表示表达式的范围，以及一个指向布尔变量 `success` 的指针。首先检查表达式是否有效，即 `p` 是否大于 `q`。然后，它会检查 `p` 是否等于 `q`，为真，则表示该 `token` 是一个数值或寄存器，接下来，它会检查是否括号匹配，即 `check_parentheses(p, q)` 是否为 `true`。如果括号匹配，则递归调用 `eval(p+1, q-1, success)` 计算括号内的表达式的值；反之，则找到主运算操作符，并计算。

(5) 在 `ui.c` 中定义 `cmd_p` 函数：参数为 `args`，它调用 `expr` 函数来计算表达式的值，并输出十进制和十六进制的结果。

测试结果如下图所示：



```
welcome to nemu
For help, type "help"
(nemu) p $pc+123
decimal: -2146434949
hex    : 0x8010007b
(nemu) p $pc
decimal: -2146435072
hex    : 0x80100000
(nemu) p -1+2
decimal: 1
hex    : 0x1
(nemu) p (6-2)/2+(3+1)*5
decimal: 22
hex    : 0x16
(nemu) p ((1-1)
expression error
```

图 2.4 算数表达式运算

## 2.3 监视点

简易调试器允许用户同时设置多个监视点，删除监视点，因此最好使用链表将监视点的信息组织起来。代码文件中已经给出了监视点定义的基本框架，在 `nemu/include/monitor/watchpoint.h` 文件中，将其完善即可，需要添加的有四个值，分别为监视点表达式 `expr`、是否改变 `valueChanged`、新旧值 `oldValue`，`nowValue`。

(1) 创建监视点：在 `watchpoint.c` 文件中，定义一个名为 `new_wp()` 的函数，用于创建一个新的监视点。传入一个表达式和它的值作为参数。使用一个数组来管理监视点。在创建监视点时，会首先检查是否存在空闲的位置。

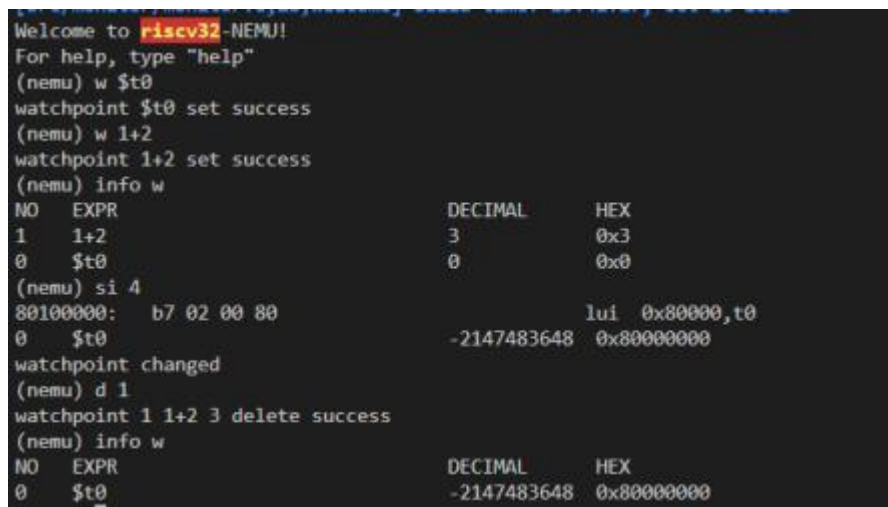
如果有空闲位置，我们将新的监视点插入到数组中，并更新相关指针。最后，将表达式的名称和值存储到新的监视点中。

(2) 删除监视点：在 `watchpoint.c` 文件中，定义一个名为 `free_wp()` 的函数，用于删除指定序号的监视点。首先检查序号的有效性和监视点数组不为空。然后，遍历数组查找对应序号的监视点。如果找到了对应的监视点，我们将它从数组中删除，并将其插入到空闲监视点链表中。最后，我们返回被删除的监视点的地址。

(3) 显示监视点：在 `watchpoint.c` 文件中，定义一个名为 `wp_display()` 的函数。用于遍历监视点数组，并显示每个监视点的信息。我们可以根据需要选择是否显示监视点的值。

(4) 更新监视点并暂停程序：在 `watchpoint.c` 文件中，定义一个名为 `wp_update()` 的函数，用于遍历监视点数组，并检查每个监视点的表达式值是否发生变化。如果发生变化，我们更新旧值并返回 `false`，否则返回 `true`。在 `cpu-exec.c` 文件中，我们调用 `wp_update()` 函数来检查监视点是否需要暂停程序的执行。如果返回 `false`，我们将程序状态设置为停止状态。

测试结果如下图所示：

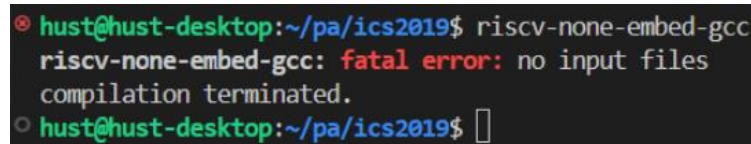


```
Welcome to riscv32-NEMU!
For help, type "help"
(nemu) w $t0
watchpoint $t0 set success
(nemu) w 1+2
watchpoint 1+2 set success
(nemu) info w
NO  EXPR          DECIMAL  HEX
1   1+2           3        0x3
0   $t0           0        0x0
(nemu) si 4
80100000: b7 02 00 80          lui  0x80000,t0
0   $t0          -2147483648 0x80000000
watchpoint changed
(nemu) d 1
watchpoint 1 1+2 3 delete success
(nemu) info w
NO  EXPR          DECIMAL  HEX
0   $t0          -2147483648 0x80000000
```

图 2.5 监视点功能展示

### 3 PA2 - 简单复杂的机器：冯诺依曼计算机系统

从 wiki 中获取交叉编译器，然后将其解压并在 ~/.bashrc 配置环境变量，再次打开终端使得配置生效，执行命令 riscv-none-embed-gcc，出现如下图 3-1 的结果，说明配置成功。



```
hust@hust-desktop:~/pa/ics2019$ riscv-none-embed-gcc
riscv-none-embed-gcc: fatal error: no input files
compilation terminated.
hust@hust-desktop:~/pa/ics2019$
```

图 3.1 riscv 测试结果

#### 3.1 运行第一个 C 程序

首先，在 nexus-am/tests/cputest 目录下执行 make ARCH=riscv32-nemu ALL=dummy run 命令，编译并运行 dummy 程序，会发现程序出现错误。通过阅读文档，并在 nexus-am/tests/cputest/build/ 目录中找到 dummy-riscv32-nemu.txt 文件，查看其反汇编代码。我们可以发现有几条指令没有实现，它们分别是 li、auipc、addi、jal、ret、lui、mv、sw、j。我们需要实现这几个指令。

(1) 为了实现指令，需要在 opcode\_table 中填写正确的译码辅助函数，执行辅助函数以及操作数宽度。通过讲义得知 pocode\_table 其实是一个索引表，位于 nemu/src/isa/riscv32/exec/exec.c 文件中，根据其下边的函数 isa\_exec()，发现是通过 opcode 的 6-2 位进行索引。查阅资料找到上述指令的 opcode，根据规则将其填入到 opcode\_table 中对应的位置。

(2) 声明译码辅助函数：将 decode.c 文件中定义的函数声明添加到 decode.h 头文件中。添加译码辅助函数：在 decode.c 文件中，已经定义了 U 型指令的译码辅助函数。接下来，我们需要添加 I 型指令和 J 型指令的译码辅助函数，解析指令的各个比特信息，并将有效信息存储到 DecodeInfo 结构体的相应成员中。例如，对于 I 型指令，它有 rs1 字段、立即数字段和 rd 字段，



译码的主要任务就是将这些字段的信息存储到 DecodeInfo 结构体对应的 Operand 成员中。

(3) 声明执行辅助函数：在 all-instr.h 文件中，我们需要添加需要编写的执行辅助函数。定义执行辅助函数：auipc指令属于计算类型的指令，在 compute.c 中定义 make\_EHelper(auipc) 函数，调用 rtl 函数实现 auipc 指令的功能。addi 指令是 I 型指令，在 compute.c 中定义。I 型指令的opcode字段很多都相同，所以在 exec.c 中的 opcode\_table 表根据指令的 opcode 来确定使用哪个译码和执行辅助函数。因此，我们可以将 opcode 为4的 I 型指令的执行辅助函数定义为 make\_EHelper(I\_opcode\_4)，并根据 I 型指令的 funtc3 和 funtc7 字段来区分不同的I型指令。例如，addi 指令的 funtc3 字段为 0，调用 rtl 函数实现 addi 指令。Jal 和 jalr 指令属于控制转移指令，在 control.c 中定义为 make\_EHelper(jal)和make\_EHelper(jalr)，调用 rtl 函数实现即可。

完成以上工作后，再次运行dummy，成功触发了好的陷阱（HIT GOOD TRAP）。测试结果如下图所示：



```
Welcome to riscv32-NEMU!  
For help, type "help"  
nemu: HIT GOOD TRAP at pc = 0x80100030  
  
[src/monitor/cpu-exec.c,34,monitor_statistic] total guest instructions = 13  
dummy
```

图 3.2 成功运行 dummy 程序

## 3.2 实现更多的指令

在3.1中，只完成了一个程序所需要的指令，接下来要完善 riscv 的其他指令：

(1) 校准指令：在 common.h 文件中，我们可以定义DIFF\_TEST宏，表示 diff-test。在 diff-test.c 文件中，编写一个名为 isa\_diff\_test\_checkregs() 的函数，该函数用于比较 ref\_r 中的32个寄存器的值与 cpu 中的32个寄存器的值是否相同。如果不相同，则返回 false。例如，在riscv32中，jalr指令需要添加 diff\_test\_skip\_dut(1, 2) 来进行校准。

(2) 添加更多的指令来运行更多的程序：在nexus-am/tests/cputest目录中有许多测试程序，我们需要添加指令，以便能够运行所有的测试。添加指

令的过程与上一节实验类似，即查看汇编代码、添加译码辅助函数、添加执行辅助函数和填写opcode\_table。

(3) 以B型指令为例：很明显B型指令应该是位于opcode\_table的一个位置，由于不同的B型指令有不同的执行译码函数，因此要给所有的B型指令br、beq、bne、blt、bge、bltu、bgeu都定义一个译码辅助函数，这些函数的原理是一致的，都是调用rtl\_jrelop()和print\_asm\_template3()函数，前者是实现条件跳转，根据条件判断是否跳转到目标地址；后者则是打印汇编指令模板。剩余的I、R型指令的辅助函数的原理和B型是差不多的，执行辅助函数的原理与B型指令是非常相似的，便不再叙述。

(4) 修改Makefile.check文件：我们需要修改Makefile.check文件，将AM项目中的程序默认编译到riscv32-nemu平台。对于大多数位于nexus-am/tests/cputest目录下的程序，添加指令就可以通过测试。对于string.c和hello-str.c两个程序，需要完成位于nexus-am/libs/klib/src目录下的stdio.c和string.c自定义库函数的实现。

(5) 在nemu目录下使用命令bash runall.sh ISA=riscv32进行一键测试。测试结果如下图所示：

```
compiling testcases...
testcases compile OK
[ add-longlong] PASS!
[ add] PASS!
[ bit] PASS!
[ bubble-sort] PASS!
[ div] PASS!
[ dummy] PASS!
[ fact] PASS!
[ fib] PASS!
[ goldbach] PASS!
[ hello-str] PASS!
[ if-else] PASS!
[ leap-year] PASS!
[ load-store] PASS!
[ matrix-mul] PASS!
[ max] PASS!
[ min3] PASS!
[ mov-c] PASS!
[ movsx] PASS!
[ mul-longlong] PASS!
[ pascal] PASS!
[ prime] PASS!
[ quick-sort] PASS!
[ recursion] PASS!
[ select-sort] PASS!
[ shift] PASS!
[ shuixianhua] PASS!
[ string] PASS!
[ sub-longlong] PASS!
[ sum] PASS!
[ switch] PASS!
[ to-lower-case] PASS!
[ unalign] PASS!
[ wanshu] PASS!
```

图 3.3 指令集测试

### 3.3 实现输入输出，运行打字小游戏

需要完成串口、时钟、键盘和VGA输入输出程序的编写。在 `common.h` 文件中，我们可以定义宏 `#define HAS_IOE` 来开启设备。

(1) 串口设备程序：框架代码已经在 `nexus-am/am/src/nemu-common/trm.c` 中提供。我们可以直接在 `nexus-am/tests/amtest/` 目录下输入命令 `make mainargs=h run` 来测试 `hello.c` 程序。

(2) 时钟设备程序：在 `nemu-timer.c` 文件中，我们定义了静态变量 `start_time` 来接收 `riscv32-nemu` 的启动时间。在 `__am_timer_init` 函数中，调用 `inl` 函数来获取地址 `RTC_ADDR` 的时间，再定义临时变量 `cur_time` 来接收 `inl` 函数读取地址 `RTC_ADDR` 的返回值，将 `uptime->lo` 设置为当前时间与启动时间的差值，即 `cur_time - start_time`，同时将 `uptime->hi` 设置为0。完成时间设备程序后，我们可以在 `nexus-am/tests/amtest/` 目录下输入命令 `make mainargs=t run` 来测试 `rtc.c` 程序。

(3) 键盘设备程序：在 `nemu-input.c` 文件中，我们使用调用 `inl` 函数来读取 `KBD_ADDR` 地址，获取键盘的活动信息，并将返回值存入 `kbd->keycode` 中，`kbd->keycode & KEYDOWN_MASK` 的真值存入 `kbd->keydown` 中，其中 `KEYDOWN_MASK` 等于 `0x8000`，是键盘的掩码。我们可以在 `nexusam/tests/amtest/` 目录下输入命令 `make mainargs=k run` 来测试 `keyboard.c` 程序，按下按钮和松开按键都会输出相应键盘的信息。

(4) VGA设备程序：在 `vga.c` 文件中的 `vga_io_handle` 函数内，我们添加代码以检测VGA设备是否发生写事件，如果是，则调用 `update_screen` 函数。在 `nemu-video.c` 文件中的 `__am_video_read` 函数中，我们初始化了一个名为 `info` 的 `_DEV_VIDEO_INFO_t` 变量的长和宽，并在 `__am_video_write` 函数中实现了对VGA设备的写入，也就是将 `pixels` 数组中的信息写入到 `fb` 数组中。同时，我们可以在 `__am_vga_init` 函数中添加相应的代码，初始化一个彩色屏幕。

上述的测试结果如下图所示





图 3.8 typing 测试

## 4 PA3 - 穿越时空的旅程：批处理系统

### 4.1 实现自陷操作及其过程

根据讲义，首先在 `nanos-lite` 中触发一次自陷操作，即注释掉 `nanos-lite/include/common.h` 中定义宏 `HAS_CTE`，利用讲义中提到一些状态寄存器，设置异常入口地址。然后便是保存上下文，在这里需要根据 `nexus-am/am/src/$ISA/nemu/trap.S` 中的代码重新组织 `_Context` 结构体。最后一步是恢复上下文，在这里需要实现一条新指令，实现指令依旧按照即可。具体实现过程如下：

(1) 在 `system.c` 文件中定义 `make_EHelper(system)` 函数。该函数根据指令的 `funct3` 字段区分 `ecall`、`sret`、`csrrw` 和 `csrrs` 指令。对于 `ecall` 和 `sret` 指令，它们的 `funct3` 字段相同，需要根据立即数字段加以区分。对于 `csrrw` 和 `csrrs` 指令，我们需要添加额外的寄存器来存储程序的状态。因此，在 `reg.h` 文件中添加了 `stvec`（存放异常入口地址）、`sepc`（存放触发异常的PC）、`sstatus`（存放处理器的状态）和 `scause`（存放触发异常的原因）等寄存器。根据不同的 `csr` 字段，`csrrw` 和 `csrrs` 指令具有不同的作用。

(2) 在 `exec.c` 文件中为 `opcode_table` 表添加 `IDEX(I, system)`，进一步完善 `opcode_table`。

(3) 在 `intr.c` 文件中，完成函数 `void raise_intr(uint32_t NO, vaddr_t epc)`，实现模拟异常响应机制。该函数将触发异常的PC存入CPU的 `sepc` 寄存器中，并将异常号存入 `scause` 寄存器中，然后使用 `rtl_j` 函数跳转到存放异常入口地址的处理程序。

(4) 重新组织 `_Context` 结构体：根据 `nexus-am/am/src/$ISA/nemu/trap.S` 的汇编指令，调整 `_Context` 结构体中成员的位置为 `uintptr_t gpr[32]`、`cause`、`status` 和 `epc`。

(5) 补充 `cte.c` 文件中的 `_Context* __am_irq_handle(_Context *c)` 函数。根据中断异常号，将事件设置为 `_EVENT_YIELD`、`_EVENT_SYSCALL`（和 `_EVENT_ERROR`。当遇到中断异常号为-1的情况时，将事件标记为 `_EVENT_YIELD`。补充 `irq.c` 文件中的 `static _Context *do_event(_Event e,`

\_Context \*c)函数。根据事件号处理不同的事件，包括\_EVENT\_YIELD、\_EVENT\_SYSCALL 和\_EVENT\_ERROR。如果要想实现自陷事件，需要处理\_EVENT\_YIELD，可以使用LOG函数输出相应的信息。

测试结果如下图所示：

```
[/home/hust/hust/ics2019/nanos-lite/src/main.c,14,main] 'Hello World!' from Nanos-lite
[/home/hust/hust/ics2019/nanos-lite/src/main.c,15,main] Build time: 15:40:37, Oct 18 2022
[/home/hust/hust/ics2019/nanos-lite/src/ramdisk.c,28,init_ramdisk] ramdisk info: start = , and = , size = ./,*,**{ bytes
[/home/hust/hust/ics2019/nanos-lite/src/device.c,81,init_device] Initializing devices...
[/home/hust/hust/ics2019/nanos-lite/src/irq.c,30,init_irq] Initializing interrupt/exception handler...
[/home/hust/hust/ics2019/nanos-lite/src/proc.c,31,init_proc] Initializing processes...
[/home/hust/hust/ics2019/nanos-lite/src/main.c,33,main] Finish initialization
[/home/hust/hust/ics2019/nanos-lite/src/irq.c,14,do_event] self trapping event
[/home/hust/hust/ics2019/nanos-lite/src/main.c,39,main] system panic: Should not reach here
```

图 4.1 自陷事件

## 4.2 实现用户程序的加载和系统调用

(1) 用户程序的加载的实现，即实现 loader 模块：在loader.c文件中定义了static uintptr\_t loader(PCB \*pcb, const char \*filename)函数，需要完善此函数。该函数的作用是将用户程序加载到正确的内存位置，然后执行用户程序。需要调用 ramdisk\_read(void\*, size\_t, size\_t) 函数来读取ramdisk文件中的数据。根据Elf\_Ehdr和Elf\_Phdr的内容，读取数据。在proc.c文件中的void init\_proc()函数中添加语句naive\_oload(NULL, NULL)将读取dummy程序，这将触发1号异常处理事件。

(2) 识别系统调用：在 nanos.c 文件中有待完成的系统调用函数。完成需要的系统调用函数实际上就是调用syscall\_(intptr\_t type, intptr\_t a0, intptr\_t a1, intptr\_t a2) 函数。系统调用号在navy-apps/libs/libos/src/syscall.h中定义，系统调用号有SYS\_exit。然后在cte.c文件中的\_\_am\_irq\_handle(\_Context \*c)函数中添加case SYS\_exit，在irq.c文件中的do\_event(\_Event e, \_Context\* c)函数中添加case \_EVENT\_SYSCALL使得do\_event函数能够识别\_EVENT\_SYSCALL，并将此事件交给do\_syscall函数处理。

(3) 实现SYS\_yield系统调用：需要在 syscall.c 文件中，完成do\_syscall(\_Context \*c)函数。首先需要实现 riscv32-nemu.h 文件下的GPR?宏，然后回到do\_syscall函数，使数组a的元素指向正确的GPR?寄存器，a[i] = c->GPRi、a[0]为系统调用号，通过switch语句对不同的系统调用号进行处理。本次处理两个系统调用号：SYS\_yield时，调用\_yield()函数实现自陷；SYS\_exit的



时, 调用\_exit()函数结束程序, 但是这里直接调用\_halt()函数结束程序。两个系统调用的返回值都设为0, 存入GPRx中。

在Nanos-lite上运行Hello World: 与之前调用dummy类似将nanoslite/Makefile中的SINGLE\_APP = \$(NAVY\_HOME)/tests/dummy 中的dummy改为hello, 再次编译运行程序即可看到连续输出的"Hello World"字符串。

(4) 实现堆区管理: 在navy-apps/libs/libos/src/nanos.c 中, 我们需要先实现 SYS\_brk 系统调用, 然后在用户层实现\_sbrk()。SYS\_brk 的系统调用实现和前边的都是一致的然后添加 nexus-am/am/src/riscv32/nemu/cte.c 中的\_\_am\_irq\_handle()函数中switch 的 case 值, 此处添加 9, 调用 do\_syscall, 然后输入命令进行测试。

测试结果如下图所示:

```
[/home/hust/hust/ics2019/nanos-lite/src/main.c,14,main] 'Hello World!' from Nanos-lite
[/home/hust/hust/ics2019/nanos-lite/src/main.c,15,main] Build time: 17:27:27, Oct 18 2022
[/home/hust/hust/ics2019/nanos-lite/src/randisk.c,28,init_randisk] randisk info: start = , end = , size = -./,*,-.0- bytes
[/home/hust/hust/ics2019/nanos-lite/src/device.c,81,init_device] Initializing devices...
[/home/hust/hust/ics2019/nanos-lite/src/irq.c,30,init_irq] Initializing interrupt/exception handler...
[/home/hust/hust/ics2019/nanos-lite/src/proc.c,31,init_proc] Initializing processes...
[/home/hust/hust/ics2019/nanos-lite/src/loader.c,52,naive_uoload] Jump to entry = 0x830000c8
[/home/hust/hust/ics2019/nanos-lite/src/irq.c,14,do_event] self trapping event
nemu: HIT GOOD TRAP at pc = 0x80100cc0
```

图 4.2 dummy 程序测试

```
Welcome to riscv32-NEMU!
For help, type "help"
[/home/hust/hust/ics2019/nanos-lite/src/main.c,14,main] 'Hello World!' from Nanos-lite
[/home/hust/hust/ics2019/nanos-lite/src/main.c,15,main] Build time: 17:37:43, Oct 18 2022
[/home/hust/hust/ics2019/nanos-lite/src/randisk.c,28,init_randisk] randisk info: start = , end = , size = -./,*,-. bytes
[/home/hust/hust/ics2019/nanos-lite/src/device.c,81,init_device] Initializing devices...
[/home/hust/hust/ics2019/nanos-lite/src/irq.c,30,init_irq] Initializing interrupt/exception handler...
[/home/hust/hust/ics2019/nanos-lite/src/proc.c,31,init_proc] Initializing processes...
[/home/hust/hust/ics2019/nanos-lite/src/loader.c,52,naive_uoload] Jump to entry = 0x83000120
Hello World!
Hello World from Navy-apps for the 2th time!
Hello World from Navy-apps for the 3th time!
Hello World from Navy-apps for the 4th time!
Hello World from Navy-apps for the 5th time!
Hello World from Navy-apps for the 6th time!
Hello World from Navy-apps for the 7th time!
Hello World from Navy-apps for the 8th time!
Hello World from Navy-apps for the 9th time!
Hello World from Navy-apps for the 10th time!
Hello World from Navy-apps for the 11th time!
Hello World from Navy-apps for the 12th time!
Hello World from Navy-apps for the 13th time!
Hello World from Navy-apps for the 14th time!
Hello World from Navy-apps for the 15th time!
```

图 4.3 hello 程序测试

```
Hello World from Navy-apps for the 1704th time!
Hello World from Navy-apps for the 1705th time!
Hello World from Navy-apps for the 1706th time!
Hello World from Navy-apps for the 1707th time!
Hello World from Navy-apps for the 1708th time!
Hello World from Navy-apps for the 1709th time!
Hello World from Navy-apps for the 1710th time!
[src/monitor/cpu-exec.c,29,monitor_statistic] total guest instructions = 91691045
make: *** [/home/hust/pa/ics2019/nexus-am/am/arch/platform/nemu.mk:27: run] Interrupt
```

图 4.3 堆管理测试



## 4.3 实现文件系统

(1) 首先，我们需要对Makefile文件进行修改。将原有的 `update: update-ramdisk-single src/syscall.h` 改为 `update: update-ramdisk-fsimg src/syscall.h`。接着，使用 `make clean` 命令清除之前生成的 `ramdisk.img` 文件，然后重新运行 `make` 命令以生成新的 `ramdisk.img` 文件，确保文件系统的更新和一致性。接下来，在 `fs.c` 文件中的 `Finfo` 结构体中添加一个成员变量 `open_offset`，用于记录文件被打开后的读写指针位置。这一改动为文件操作提供了必要的状态管理支持。

(2) 实现基本的文件操作，包括 `fs_open`、`fs_read`、`fs_write`、`fs_lseek` 和 `fs_close`，需要注意的是，我们正在实现一个简易文件系统，因此如果 `fs_open` 未能找到文件，则需要使用 `assert` 断言来终止程序的运行，以确保文件系统的健壮性。在实现文件读写操作时，可以使用 `ramdisk_read` 和 `ramdisk_write` 函数来对 `ramdisk.img` 文件进行读写。在调用这两个函数之前，需要先判断相应的读写函数指针是否存在。如果存在，则调用相应的读写函数指针进行操作。由于文件大小是固定的，必须特别注意偏移量不要超过文件的边界。如果偏移量超出文件边界，需要将其限制在文件边界处。对于 `fs_write` 函数中的标准输出和标准错误，我们需要使用 `_putc` 函数进行输出。而对于标准输入、标准输出和标准错误这三个特殊文件，其他文件操作可以直接忽略。

(3) 接下来，我们需要为文件系统添加系统调用，包括 `SYS_open`、`SYS_read`、`SYS_close` 和 `SYS_lseek`。这些系统调用的实现可以参考 PA3.2(2)(4) 中的添加过程。此外，还需要修改 `loader` 函数使用文件的方式。之前的做法是在 `loader.c` 文件的 `loader` 函数中直接调用 `ramdisk_read()` 函数来加载用户程序。然而，随着程序数量的增加，这种方式变得难以管理。因此，我们可以修改 `loader` 函数，使用上述实现的 `fs_xxx` 函数来加载用户程序。修改完成后，在 `proc.c` 文件的 `init_proc` 函数中调用 `naive_uoload(PCB *pcb, const char *filename)` 函数来打开程序。只需修改 `filename` 参数即可运行不同的程序。

(4) 最后，完成文件系统的实现后，我们可以进行测试。通过修改 `naive_uoload(NULL, "/bin/text")` 来加载程序，并再次运行。终端将输出 "PASS!!!" 的信息，表明文件系统的实现和集成工作已成功完成。

测试结果如下图所示：

```

for help, type "help"
[/home/hust/hust/ics2019/nanos-lite/src/main.c,14,main] 'Hello World!' from Nanos-lite
[/home/hust/hust/ics2019/nanos-lite/src/main.c,15,main] Build time: 18:27:08, Oct 18 2022
[/home/hust/hust/ics2019/nanos-lite/src/ramdisk.c,28,init_ramdisk] ramdisk info: start = , end = , size = -.
/,*,..)-. bytes
[/home/hust/hust/ics2019/nanos-lite/src/device.c,81,init_device] Initializing devices...
[/home/hust/hust/ics2019/nanos-lite/src/irq.c,30,init_irq] Initializing interrupt/exception handler...
[/home/hust/hust/ics2019/nanos-lite/src/proc.c,31,init_proc] Initializing processes...
[/home/hust/hust/ics2019/nanos-lite/src/loader.c,52,naive_uoload] Jump to entry = 0x830002f4
PASS!!!
nemu: HIT GOOD TRAP at pc = 0x80100db4

```

图 4.4 text 程序测试

## 4.4 运行仙剑奇侠传

(1) 串口抽象为文件：在device.c文件中实现 serial\_write(const void \*buf, size\_t offset, size\_t len) 函数：使用\_putc函数将buf中的字符逐个输出到串口修改fs.c文件中的file\_table表：将标准输出和标准错误的write 指针修改为(WriteFn)serial\_write。在fs\_write()函数中：对于文件描述符为1和2的情况，用文件对象的write指针指向的函数，实际上会调用serial\_write()函数进行输出。

(2) 设备输入抽象为文件：将输入设备（如时钟和键盘）抽象为文件/dev/events：在device.c中实现 events\_read(void \*buf, size\_t offset, size\_t len) 函数：调用read\_key()函数读取键盘信息，并将其存储在key变量中。如果key & KEY\_DOWN\_MASK不为0，表示键盘按下，将键盘信息写入buf。如果没有键盘活动，则将时间信息 buf。在 fs.c 文件的 file\_table 表中添加："/dev/events", 0, 0, 0, (ReadFn)events\_read, (WriteFn)invalid\_write，修改init\_proc()函数中的naive\_uoload(NULL, "/bin/events")，编译并运行后，程序将输出时间和按键事件。

(3) VGA显存抽象为文件将显存抽象为文件/dev/fb：在device.c中实现 fb\_write(const void \*buf, size\_t offset, size\_t len)函数：根据偏移量计算坐标（y = offset / 屏幕宽度，x = offset % 屏幕宽度），调用draw\_rect()函数进行绘制，并返回长度len，在fs.c文件的file\_table表中添加："/dev/fb", 0, 0, 0, (ReadFn)invalid\_read, (WriteFn)fb\_write。

(4) 实现显存刷新机制：在device.c中实现 fbsync\_write(const void \*buf, size\_t offset, size\_t len)函数：直接调用draw\_sync()函数，并返回长度len。在fs.c文件的file\_table表中添加："/dev/fbsync", 0, 0, 0, (ReadFn)invalid\_read, (WriteFn)fbsync\_write 获取屏幕信息：在device.c中实现 dispinfo\_read(void \*buf, size\_t offset, size\_t len) 函数：在init\_device()函数中将屏幕的宽度和高度写入dispinfo数组。在dispinfo\_read()函数中将dispinfo数组写入buf。在fs.c文件的

file\_table表中添加: `"/proc/dispinfo", 128, 0, 0, (ReadFn)dispinfo_read, (WriteFn)invalid_write`, 修改init\_proc()函数中的naive\_uoload(NULL, `"/bin/bmptest"`), 编译并运行后, 屏幕上将显示"logo N".

(5) 在NEMU中运行仙剑奇侠传: 下载解压后将其中的 pal 文件夹拷贝到 navy-apps/fsimg/share/games/ 目录下。修改init\_proc() 函数中的naive\_uoload (NULL, `"/bin/pal"`), 重新编译并运行, 即可进入仙剑奇侠传游戏。

(6) 实现批处理系统实现SYS\_execve系统调用: 在nanos.c文件中实现SYS\_execve系统调用。在cte.c中识别该系统调用。在syscall.c中处理SYS\_execve系统调用: 在SYS\_execve处调用naive\_uoload(NULL, a[0])加载菜单选择的程序。在SYS\_exit处调用naive\_uoload(NULL, `"/bin/init"`)重新加载开始菜单。在fs.c文件的file\_table表中添加: `"/dev/tty", 0, 0, 0, (ReadFn)invalid_read, (WriteFn)serial_write}`修改proc.c文件中的 naive\_uoload(NULL, `"/bin/init"`), 重新编译并运行后, 将显示开始菜单

测试结果如下图所示:

```

Welcome to riscv32-NEMU!
For help, type 'help'
/home/hust/hust/ics2019/nanos-lite/src/main.c:14,main: 'Hello World!' from Nanos-lite
/home/hust/hust/ics2019/nanos-lite/src/main.c:15,main: Build time: 19:12:07, Oct 18 2022
/home/hust/hust/ics2019/nanos-lite/src/ramdisk.c:78,init_ramdisk: ramdisk info: start = , end = , size = ,/*,...)) bytes
/home/hust/hust/ics2019/nanos-lite/src/device.c:82,init_device: Initializing devices...
/home/hust/hust/ics2019/nanos-lite/src/irq.c:30,init_irq: Initializing interrupt/exception handler...
/home/hust/hust/ics2019/nanos-lite/src/proc.c:31,init_proc: Initializing processes...
/home/hust/hust/ics2019/nanos-lite/src/loader.c:52,naive_uoload: Jump to entry = 0x1000100
Start to receive events...
receive event: kd A
receive event: kd D
receive event: ku D
receive event: kd A
receive event: ku A
receive event: kd D
receive event: ku D
receive time event for the 1024th time: t 55
receive time event for the 2048th time: t 91
receive time event for the 3072th time: t 133
receive time event for the 4096th time: t 172
receive event: kd D
receive time event for the 5120th time: t 212
receive event: kd A
receive event: ku D
receive time event for the 6144th time: t 269
receive time event for the 7168th time: t 307
receive event: ku A
```

图 4.5 events 程序测试



图 4.6 bmptest 程序测试



图 4.7 仙剑奇侠传游戏初始化画面



图 4.8 仙剑奇侠传游戏对话画面

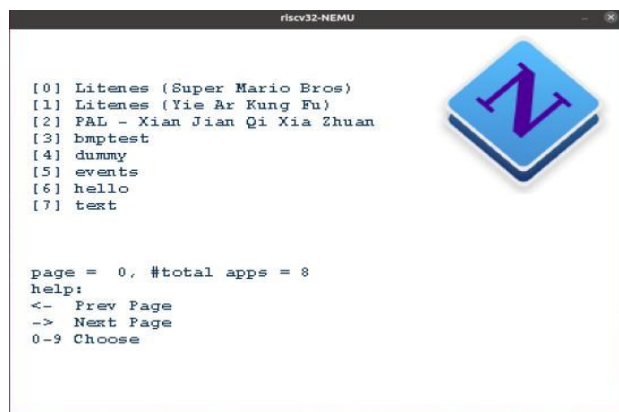


图 4.9 开始菜单

## 5 总结与心得

通过参与PA项目，我深刻体会到了它的趣味性和挑战性。PA项目涵盖了计算机系统的多个核心知识点，不仅拓宽了我的技术视野，还提升了我的系统开发能力，让我学会了从系统的角度去分析和解决问题。为我提供了一个难得的实践机会，让我在动手实践中收获了宝贵的经验。

在PA1阶段，我实现了一个简易调试器，这是NEMU中非常重要的基础设施。通过实现调试器，我不仅提高了调试效率，还熟悉了框架代码的结构和运行机制，为后续的挑战奠定了坚实的基础。简易调试器的主要功能包括帮助指令、继续运行指令、退出、单步执行指令、打印程序状态、表达式求值、扫描内存、设置监视点和删除监视点。其中，表达式求值是最具挑战性的部分，涉及识别tokens、处理解引用和负数等运算符，以及编写求值运算函数eval()。这一过程让我对编译原理和表达式解析有了更深入的理解。

在PA2阶段，核心任务是实现RISC-V指令集。这一阶段的开发流程离不开指令的设计，因此在实验开始前，我建议先学习相关的书籍，并深入理解NEMU框架的设计思想。我的经验是先完成diff-test，然后再着手指令的设计。实验中涉及的指令非常多，需要查阅RISC-V指令集手册并逐一实现。指令设计不仅包括译码辅助函数和执行辅助函数的编写，还需要深刻理解这两种函数的作用和实现原理。在输入输出设备方面，我主要完成了VGA设备的设计，这让我对硬件与软件的交互有了更直观的认识。

在PA3阶段，核心内容是实现系统调用和文件系统。这一部分的难度较高，涉及汇编语言、联合编译和ELF文件格式等知识。重难点包括添加自陷指令和相应的CSR寄存器、理解系统调用的完整流程、编写loader函数，以及实现文件系统将设备抽象为文件，并通过读写函数实现对设备文件的操作。这一阶段让我对操作系统的核心机制有了更深入的理解，尤其是系统调用和文件系统的实现过程，让我认识到操作系统是如何管理资源和提供服务的。

总的来说，PA项目具有一定的难度，需要投入大量时间阅读相关书籍和框架代码。只有深入理解框架的设计思想和运行机制，才能顺利完成实验内容。通过PA项目，我不仅掌握了计算机系统的核心知识，还培养了解决复杂问题的能力。这段经历让我对计算机系统的理解更加全面，也让我对未来的学习和研究充满了信心。

学号: U202012663  
姓名: 李宝宁

图 5.1 电子签名